50 AI-Powered Productivity Hacks for Web Developers

Prompt 1: Code Review Checker

Act as a code review assistant. Review the following code snippet (example provided in the prompt) and suggest improvements, including suggestions for better coding practices, performance optimizations, and potential security vulnerabilities.

Code snippet:

```javascript
const database = require('./database');

module.exports = async function getPost(postId) {
const query = `SELECT * FROM posts WHERE post_id = $1`;
const result = await database.pool.query(query, [postId]);

if (result.rows.length === 0) {
return null;
}

return result.rows[0];
};
```

Prompt 2: Frontend Optimizer

Act as a frontend optimization expert. Provide suggestions for optimizing the following CSS file (example provided in the prompt) for better loading speed, including minification, compression, and code splitting.

CSS file:

```css
body {
background-color: #f2f2f2;
font-family: Arial, sans-serif;
}

.container {
max-width: 800px;
margin: 40px auto;
padding: 20px;
background-color: #ffffff;
border: 1px solid #dddddd;
border-radius: 10px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.header {
```

```
font-size: 24px;

font-weight: bold;

color: #00698f;

margin-bottom: 20px;

}

button {

background-color: #00698f;

color: #ffffff;

padding: 10px 20px;

border: none;

border-radius: 5px;

cursor: pointer;

}
```

Prompt 3: Accessibility Auditor

Act as an accessibility auditor. Analyze the following HTML code (example provided in the prompt) for accessibility issues, including keyboard navigation, screen reader compatibility, and semantic HTML structure.

HTML code:

```html
<div class="nav">
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">About</a></li>
<li><a href="#">Contact</a></li>
</ul>
</div>

<div class="main-content">
<h1>Welcome to our website!</h1>
<p>This is some sample content.</p>
<button>Click me!</button>
</div>
```

Prompt 4: Performance Profiler

Act as a performance profiler. Use a CPU profiler to analyze the following code snippet (example provided in the prompt) and identify performance bottlenecks, including areas for optimization.

Code snippet:

```javascript

const fetchPosts = async () => {

const response = await fetch('/api/posts');

const data = await response.json();

data.forEach(async (post) => {

const postContent = await fetch(`/api/posts/${post.id}`);

const postData = await postContent.json();

console.log(postData);

});

};

```

Prompt 5: Error Analyst

Act as an error analyst. Analyze the following error message (example provided in the prompt) to identify the root cause of the issue, including possible solutions.

Error message:

```

Uncaught TypeError: Cannot read property 'posts' of undefined

```

Prompt 6: Code Formatter

Act as a code formatter. Format the following code snippet (example provided in the prompt) according to the Prettier style guide.

Code snippet:

```javascript
const user = {
name: 'John Doe',
email: 'john@example.com',
isAdmin: true
};

function greetUser() {
console.log(`Hello, ${user.name}!`);
}
```

Prompt 7: Code Commenter

Act as a code commentator. Write high-quality comments for the following code snippet (example provided in the prompt), including explanations for each section and variable names.

Code snippet:

```javascript

export function calculateTotalPrice(items) {

let totalPrice = 0;

items.forEach((item) => {

const itemPrice = item.price * item.quantity;

totalPrice += itemPrice;

});

return totalPrice;

}

```

Prompt 8: Code Searcher

Act as a code searcher. Search for instances of a specific function call (example provided in the prompt) throughout the entire codebase and report the results, including function signatures and file paths.

Function call:

```javascript

import { fetchPost } from './api/posts';

```

Prompt 9: Code Migrator

Act as a code migrator. Migrate the following code snippet (example provided in the prompt) from a previous version of JavaScript to a newer version.

Code snippet:

```javascript
function add(a, b) {
return a + b;
}
```

Prompt 10: API Documenter

Act as an API documenter. Generate API documentation for the following API endpoint (example provided in the prompt), including descriptions, parameter details, and expected response formats.

API endpoint:

```javascript
GET /api/posts/:id
```

Prompt 11: Code Generator

Act as a code generator. Generate a full CRUD (Create, Read, Update, Delete) API for a Post model, including endpoint descriptions and API documentation.

Prompt 12: Bug Hunter

Act as a bug hunter. Analyze the following bug report (example provided in the prompt) and identify the root cause of the issue, including possible solutions.

Bug report:

```javascript

Unexpected token 'EOF' in JSON at position '2'

```

Prompt 13: Feature Request Analyst

Act as a feature request analyst. Analyze the following feature request (example provided in the prompt) and provide a prioritized list of implementation steps, including technical requirements and timelines.

Feature request:

```javascript

As a user, I want to be able to save a list of favorite posts, so I can easily access them later.

```

Prompt 14: Accessibility Test

Act as an accessibility tester. Run accessibility tests on the following web page (example provided in the prompt) and report any issues found, including suggestions for improvements.

Web page:

```html
<!DOCTYPE html>
<html>
<head>
<title>Example Web Page</title>
</head>
<body>
<h1>Welcome to our website!</h1>
<p>This is some sample content.</p>
</body>
</html>
```

Prompt 15: Code Optimizer

Act as a code optimizer. Optimize the following code snippet (example provided in the prompt) for production use, including minification, compression, and code splitting.

Code snippet:

```javascript
const postContent = fetch(`https://api.example.com/posts/${postId}`)
.then((response) => {
return response.json();
})
.then((data) => {
console.log(data);
});
```

Prompt 16: Error Handler

Act as an error handler. Analyze the following error message (example provided in the prompt) and provide a suggested error handling strategy, including ways to log and display error messages.

Error message:

```javascript
TypeError: Cannot read property 'posts' of undefined
```

Prompt 17: Security Auditor

Act as a security auditor. Analyze the following code snippet (example provided in the prompt) for potential security vulnerabilities, including SQL injection and cross-site scripting (XSS) attacks.

Code snippet:

```javascript
const query = `SELECT * FROM posts WHERE id = ${postId}`;
const result = await database.pool.query(query);
```

Prompt 18: Code Reviewer

Act as a code reviewer. Review the following code snippet (example provided in the prompt) and provide feedback on coding practices, performance, and security.

Code snippet:

```javascript
for (let i = 0; i < 10; i++) {
const post = await fetch(`https://api.example.com/posts/${i}`);
await post.json();
}
```

Prompt 19: API Analyzer

Act as an API analyzer. Analyze the following API endpoint (example provided in the prompt) and provide recommendations for rate limiting, API keys, and OAuth 2.0 authorization.

API endpoint:

```javascript
GET /api/posts/
```

Prompt 20: Code Comparator

Act as a code comparator. Compare the following two code snippets (example provided in the prompt) and report any differences, including variable names, function signatures, and logical flow.

Code snippet 1:

```javascript
const posts = [];
for (const post of postsApi) {
posts.push(post);
}
```

Code snippet 2:

```javascript
```

```
const posts = postsApi.map((post) => post);
```

Prompt 21: Code Formatter

Act as a code formatter. Format the following code snippet (example provided in the prompt) according to the Prettier style guide.

Code snippet:

```javascript
for (let i = 0; i < 10; i++) {
const post = await fetch(`https://api.example.com/posts/${i}`);
const postJson = await post.json();
console.log(postJson);
}
```

Prompt 22: Code Searcher

Act as a code searcher. Search for instances of a specific function call (example provided in the prompt) throughout the entire codebase and report the results, including function signatures and file paths.

Function call:

```javascript
```

```
import { fetchPost } from './api/posts';
```

Prompt 23: Code Analyzer

Act as a code analyzer. Analyze the following code snippet (example provided in the prompt) for performance issues, including function overhead and memory usage.

Code snippet:

```javascript
for (let i = 0; i < 10000; i++) {
const post = await fetch(`https://api.example.com/posts/${i}`);
await post.json();
console.log(post);
}
```

Prompt 24: Code Minifier

Act as a code minifier. Minify the following code snippet (example provided in the prompt) for production use.

Code snippet:

```javascript
```

```
const posts = [];

for (const post of postsApi) {

posts.push(post);

}
```

Prompt 25: Code Compressor

Act as a code compressor. Compress the following code snippet (example provided in the prompt) using a lossless compression algorithm.

Code snippet:

```javascript
for (let i = 0; i < 10; i++) {

const post = await fetch(`https://api.example.com/posts/${i}`);

const postJson = await post.json();

console.log(postJson);

}
```

Prompt 26: Code Optimizer

Act as a code optimizer. Optimize the following code snippet (example provided in the prompt) for production use, including minification, compression, and code splitting.

Code snippet:

```javascript
const postContent = fetch(`https://api.example.com/posts/${postId}`)
.then((response) => {
return response.json();
})
.then((data) => {
console.log(data);
});
```

Prompt 27: Code Splitter

Act as a code splitter. Split the following code snippet (example provided in the prompt) into separate files, including a main entry point and reusable functions.

Code snippet:

```javascript
function getPost(id) {
return fetch(`https://api.example.com/posts/${id}`)
.then((response) => response.json())
.then((data) => data);
```

```
}

(async () => {

const post = await getPost('123');

console.log(post);

})();
```

Prompt 28: Code Generator

Act as a code generator. Generate a basic CRUD (Create, Read, Update, Delete) API for a Post model, including API endpoint descriptions and API documentation.

Prompt 29: Code Commenter

Act as a code commentator. Write high-quality comments for the following code snippet (example provided in the prompt), including explanations for each section and variable names.

Code snippet:

```javascript
export function calculateTotalPrice(items) {

let totalPrice = 0;

items.forEach((item) => {

const itemPrice = item.price * item.quantity;
```

```
totalPrice += itemPrice;

});

return totalPrice;

}
```

Prompt 30: Security Scanner

Act as a security scanner. Analyze the following code snippet (example provided in the prompt) for potential security vulnerabilities, including SQL injection and cross-site scripting (XSS) attacks.

Code snippet:

```javascript
const query = `SELECT * FROM posts WHERE id = ${postId}`;

const result = await database.pool.query(query);
```

Prompt 31: Code Reviewer

Act as a code reviewer. Review the following code snippet (example provided in the prompt) and provide feedback on coding practices, performance, and security.

Code snippet:

```javascript
```

```
for (let i = 0; i < 10; i++) {

const post = await fetch(`https://api.example.com/posts/${i}`);

await post.json();

}
```

Prompt 32: Code Comparator

Act as a code comparator. Compare the following two code snippets (example provided in the prompt) and report any differences, including variable names, function signatures, and logical flow.

Code snippet 1:

```javascript
const posts = [];

for (const post of postsApi) {

posts.push(post);

}
```

Code snippet 2:

```javascript
const posts = postsApi.map((post) => post);
```

```

## Prompt 33: Code Analyzer

Act as a code analyzer. Analyze the following code snippet (example provided in the prompt) for performance issues, including function overhead and memory usage.

Code snippet:

```javascript

for (let i = 0; i < 10000; i++) {

const post = await fetch(`https://api.example.com/posts/${i}`);

await post.json();

console.log(post);

}

```

Prompt 34: Code Searcher

Act as a code searcher. Search for instances of a specific function call (example provided in the prompt) throughout the entire codebase and report the results, including function signatures and file paths.

Function call:

```javascript

import { fetchPost } from './api/posts';

```

Prompt 35: API Analyzer

Act as an API analyzer. Analyze the following API endpoint (example provided in the prompt) and provide recommendations for rate limiting, API keys, and OAuth 2.0 authorization.

API endpoint:

```javascript

GET /api/posts/

```

Prompt 36: Code Generator

Act as a code generator. Generate a full CRUD (Create, Read, Update, Delete) API for a Post model, including API endpoint descriptions and API documentation.

Prompt 37: Code Formatter

Act as a code formatter. Format the following code snippet (example provided in the prompt) according to the Prettier style guide.

Code snippet:

```javascript

for (let i = 0; i < 10; i++) {

```
const post = await fetch(`https://api.example.com/posts/${i}`);

const postJson = await post.json();

console.log(postJson);

}
```

Prompt 38: Code Minifier

Act as a code minifier. Minify the following code snippet (example provided in the prompt) for production use.

Code snippet:

```javascript
const posts = [];

for (const post of postsApi) {

posts.push(post);

}
```

Prompt 39: Code Compressor

Act as a code compressor. Compress the following code snippet (example provided in the prompt) using a lossless compression algorithm.

Code snippet:

```javascript

for (let i = 0; i < 10; i++) {

const post = await fetch(`https://api.example.com/posts/${i}`);

const postJson = await post.json();

console.log(postJson);

}

```

Prompt 40: Code Optimizer

Act as a code optimizer. Optimize the following code snippet (example provided in the prompt) for production use, including minification, compression, and code splitting.

Code snippet:

```javascript

const postContent = fetch(`https://api.example.com/posts/${postId}`)

.then((response) => {

return response.json();

})

.then((data) => {

console.log(data);
```

});

```

## Prompt 41: Code Splitter

Act as a code splitter. Split the following code snippet (example provided in the prompt) into separate files, including a main entry point and reusable functions.

Code snippet:

```javascript

function getPost(id) {

return fetch(`https://api.example.com/posts/${id}`)

.then((response) => response.json())

.then((data) => data);

}

(async () => {

const post = await getPost('123');

console.log(post);

})();

```

Prompt 42: Code Generator
```

Act as a code generator. Generate a basic CRUD (Create, Read, Update, Delete) API for a Post model, including API endpoint descriptions and API documentation.

Prompt 43: Code Commenter

Act as a code commentator. Write high-quality comments for the following code snippet (example provided in the prompt), including explanations for each section and variable names.

Code snippet:

```javascript
export function calculateTotalPrice(items) {
let totalPrice = 0;

items.forEach((item) => {
const itemPrice = item.price * item.quantity;
totalPrice += itemPrice;
});

return totalPrice;
}
```

Prompt 44: Security Scanner

Act as a security scanner. Analyze the following code snippet (example provided in the prompt) for potential security vulnerabilities, including SQL injection and cross-site scripting (XSS) attacks.

Code snippet:

```javascript
const query = `SELECT * FROM posts WHERE id = ${postId}`;
const result = await database.pool.query(query);
```

Prompt 45: Code Reviewer

Act as a code reviewer. Review the following code snippet (example provided in the prompt) and provide feedback on coding practices, performance, and security.

Code snippet:

```javascript
for (let i = 0; i < 10; i++) {
const post = await fetch(`https://api.example.com/posts/${i}`);
await post.json();
}
```

Prompt 46: Code Comparator

Act as a code comparator. Compare the following two code snippets (example provided in the prompt) and report any differences, including variable names, function signatures, and logical flow.

Code snippet 1:

```javascript
const posts = [];
for (const post of postsApi) {
posts.push(post);
}
```

Code snippet 2:

```javascript
const posts = postsApi.map((post) => post);
```

Prompt 47: API Analyzer

Act as an API analyzer. Analyze the following API endpoint (example provided in the prompt) and provide recommendations for rate limiting, API keys, and OAuth 2.0 authorization.

API endpoint:

```javascript
```

GET /api/posts/

```

## Prompt 48: Code Generator

Act as a code generator. Generate a full CRUD (Create, Read, Update, Delete) API for a Post model, including API endpoint descriptions and API documentation.

## Prompt 49: Code Formatter

Act as
```